Multiplayer Game Platform — Initial Project Plan

Developed by Bogo Solutions

Overview

Three one-week sprints covering the full implementation of the platform as designed in the architecture document. Each sprint is organized by sub-team: **Platform Core**, **Client/UI**, **Rules & Validation**, and **Quality, Testing & Integration**.

Sprint 1 builds the foundation. Sprint 2 implements the core platform features. Sprint 3 integrates everything, closes out stats and notifications, and prepares the demo.

Branch naming convention: Each backlog ID corresponds to a feature branch of the same name, e.g., `PC-NET` → branch `feat/PC-NET`. All branches merge into `develop`; `develop` merges into `main` at sprint boundaries via a reviewed merge request.

Table of Contents

1. Sprint 1 — Foundation & Core Abstractions
2. Sprint 2 — Core Platform Features
3. Sprint 3 — Integration, Notifications & Demo
4. Dependency Summary
5. Product Backlog

Sprint 1 — Foundation & Core Abstractions (Week 1)

Goal: Establish the project skeleton, all shared interfaces and data models, stubs, and the TCP network boundary so every sub-team can work in parallel from Week 2 onward.

Platform Core

Task	Branch (Backlog ID)
Project structure setup (`ui`, `platform`, `game`, `model` packages)	PC-SETUP
`NETWORK` stub class (TCP boundary, `send` / `receive` over socket)	PC-NET
`ClientReceiver` skeleton (listen loop, message type dispatch)	PC-CLIENT

Task	Branch (Backlog ID)
`ClientCore` skeleton (`send` , `receive` , `check` stubs)	PC-CLIENT
`Database` stub (`storeData` , `retrieveData`)	PC-DB
`CredentialsDatabase` stub (login boolean, register)	PC-AUTH
`ServerCore` skeleton (`send` , `receive` , `routeContent` stubs)	PC-SERVER
Define `GameStateUpdate` event class	PC-EVENTS
Define `GameStatus` enum	PC-EVENTS
`EventBus` skeleton (`publish` , `subscribe`)	PC-EVENTS

Client/UI

Task	Branch (Backlog ID)
`WelcomePage`	UI-AUTH
`AuthenticationScreen`	UI-AUTH
`LoginPage`	UI-AUTH
`State` enum (login/register error states)	UI-AUTH

Rules & Validation

Task	Branch (Backlog ID)
`Game` interface (`startGame` , `handleAction` , `validateAction` , `getGameState` , `getGameUpdate` , `isGameOver`)	GAME-ABSTRACT
`RuleValidator` interface (`validate` , `getValidActions`)	GAME-ABSTRACT
`PlayerAction` abstraction (`playerId` , `actionType`)	GAME-ABSTRACT
`clientGame` interface (`validate(move): Bool`) — client-side validation hook for `ClientCore`	GAME-ABSTRACT
`Player` domain model	GAME-MODEL
`GeneralStats` class	GAME-MODEL

Task	Branch (Backlog ID)
`Rank` enum	GAME-MODEL
`PlayerStats` class	GAME-MODEL
`Settings` class	GAME-MODEL

Quality, Testing & Integration

Task	Branch (Backlog ID)
GitLab repository setup	QA-SETUP
README (build + run instructions)	QA-SETUP
Branching conventions documented	QA-SETUP
Test infrastructure setup (JUnit, test runner)	QA-SETUP
Unit tests: `Player`	QA-TESTS
Unit tests: `GeneralStats`	QA-TESTS
Unit tests: `CredentialsDatabase` stub	QA-TESTS
Unit tests: `NETWORK` stub send/receive	QA-TESTS

Sprint 2 — Core Platform Features (Week 2)

Goal: Implement the full request-routing pipeline through `ClientCore → NETWORK → ServerCore`, wire up `GameLobby`, `matchmaking`, `Chat`, and the `Game` service. Implement both game implementations and the primary UI screens. By end of this sprint the lobby → game start → turn progression flow should be runnable end-to-end.

Platform Core

Task	Branch (Backlog ID)
`ClientCore` — full implementation (local request handling, TCP dispatch, `clientGame.check` integration)	PC-CLIENT
`ClientReceiver` — full implementation (server-push listener, update routing to UI)	PC-CLIENT

Task	Branch (Backlog ID)
`ServerCore` — full routing (`receiveMessage` , `routeContent` to `GameLobby` , `Chat` , `Database`)	PC-SERVER
`SessionManager` (create, store, invalidate session tokens)	PC-AUTH
`GameLobby` service (`receiveContent` , `requestJoin` , display game info, route to matchmaking)	PC-LOBBY
`matchmaking` service (queue management, match-found routing, new `Game` instance creation)	PC-MATCH
`Chat` service (`receiveContent` → `sendMessage` , user list `List<int>` management)	PC-CHAT
`Game` service — skeleton (turn update routing, move dispatch to `GameEngine` , state broadcast stubs)	PC-SESSION
`GameEngine` interface (`validateMove(move): Boolean` , `updateGame(move)`)	PC-SESSION
`Leaderboard` service — skeleton (`logGameResult` , `updatePlayer`)	PC-LEADERBOARD

Client/UI

Task	Branch (Backlog ID)
`HomePage`	UI-HOME
`MainGameDashboard`	UI-HOME
`ProfileDashboard`	UI-PROFILE
`PlayGame`	UI-GAME
`GameGUI`	UI-GAME
`ChatboxGUI`	UI-GAME

Rules & Validation

Task	Branch (Backlog ID)
`TicTacToeGame` (implements `Game`)	GAME-TTT
`TicTacToePlayerAction` (implements `PlayerAction`)	GAME-TTT

Task	Branch (Backlog ID)
`TicTacToeRuleValidator` (implements `RuleValidator`)	GAME-TTT
`TicTacToeGameState` (implements `GameState`)	GAME-TTT
`ConnectFourGame` (implements `Game`)	GAME-C4
`ConnectFourPlayerAction` (implements `PlayerAction`)	GAME-C4
`ConnectFourRuleValidator` (implements `RuleValidator`)	GAME-C4
`ConnectFourGameState` (implements `GameState`)	GAME-C4

Quality, Testing & Integration

Task	Branch (Backlog ID)
Merge Sprint 1 branches into `develop`	QA-RELEASE
End-to-end smoke test (launch, login stub, reach dashboard)	QA-RELEASE
Architecture doc update	QA-SETUP
Unit tests: `TicTacToeRuleValidator`	QA-TESTS
Unit tests: `ConnectFourRuleValidator`	QA-TESTS
Unit tests: `GameLobby`	QA-TESTS
Unit tests: `matchmaking`	QA-TESTS
Unit tests: `ServerCore` routing	QA-TESTS
Integration test: login flow (`ClientCore` → `NETWORK` → `ServerCore` → `CredentialsDatabase` stub)	QA-TESTS
Integration test: lobby join flow	QA-TESTS

Sprint 3 — Integration, Notifications & Demo (Week 3)

Goal: Complete the move pipeline and notification system end-to-end (`GameSessionManager` → `EventBus` → `ServerCore` → `ClientReceiver` → UI). Close out stats, leaderboard, and all remaining UI screens. Prepare and verify the final demo.

Platform Core

Task	Branch (Backlog ID)
`Game` service — full implementation (validated move pipeline, game state broadcast to both players via `ServerCore`, local chat, return-to-lobby)	PC-SESSION
`GameSessionManager` — full implementation (move pipeline: `GameEngine.validateMove` → `updateGame` → check end conditions → publish `GameStateUpdate` to `EventBus`)	PC-SESSION
`EventBus` — full publish/subscribe wiring (`GameSessionManager` publishes, `ServerCore` subscribes and routes to `ClientReceiver`)	PC-EVENTS
`Leaderboard` service — full implementation (`getTopPlayersByRank`, MMR recalculation via `updatePlayer`)	PC-LEADERBOARD
`StatsManager` (aggregates `GeneralStats` + `Rank`, persists via `Database` stub)	GAME-MODEL

Client/UI

Task	Branch (Backlog ID)
`Settings` UI	UI-PROFILE
Win/loss/tie result screen	UI-GAME
Turn alert display (receives `GameStateUpdate` via `ClientReceiver`)	UI-GAME
Player statistics view	UI-PROFILE
Leaderboard view	UI-HOME
UI polish & full navigation wiring	UI-HOME

Rules & Validation

Task	Branch (Backlog ID)
Win detection — Tic-Tac-Toe (3-in-a-row)	GAME-TTT
Win detection — Connect Four (4-in-a-row, all directions)	GAME-C4
`clientGame` implementation for Tic-Tac-Toe (wired into `ClientCore.check`)	GAME-TTT
`clientGame` implementation for Connect Four (wired into `ClientCore.check`)	GAME-C4
Validation failure reason propagation (`ValidationResult.failureReason` surfaces to UI via `GameStateUpdate`)	GAME-ABSTRACT

Quality, Testing & Integration

Task	Branch (Backlog ID)
Merge Sprint 2 & 3 branches into `develop`, then `main`	QA-RELEASE
End-to-end demo run	QA-RELEASE
Demo script verification (scripted walkthrough of all platform flows)	QA-RELEASE
Final README update	QA-SETUP
Architecture doc final revision	QA-SETUP
Stub documentation (interface contracts for `NETWORK`, `Database`, `CredentialsDatabase`)	QA-SETUP
Integration test: full game flow (lobby → game start → turns → game over)	QA-TESTS
Integration test: turn notification (`GameStateUpdate` routed to correct player's screen)	QA-TESTS
Integration test: game-over stats update (`StatsManager` → `Database` stub)	QA-TESTS
Unit tests: `StatsManager`	QA-TESTS
Unit tests: `GameSessionManager`	QA-TESTS
Unit tests: win detection (Tic-Tac-Toe, Connect Four)	QA-TESTS
Unit tests: `EventBus` publish/subscribe	QA-TESTS

Dependency Summary

Sprint 2 / 3 Task	Depends On (Sprint 1)
`ClientCore` full impl	`PC-NET` (NETWORK stub), `GAME-ABSTRACT` (clientGame interface), `PC-CLIENT` skeleton
`ClientReceiver` full impl	`PC-NET` (NETWORK stub), `PC-CLIENT` skeleton
`ServerCore` full routing	`PC-NET`, `PC-DB`, `PC-AUTH` stubs
`SessionManager`	`PC-AUTH` (CredentialsDatabase stub), `GAME-MODEL` (Player model)
`GameLobby`	`PC-SERVER` skeleton, `PC-MATCH` skeleton
`matchmaking`	`GAME-MODEL` (Player), `PC-DB` (Database stub)
`Chat`	`PC-LOBBY` (GameLobby service)

Sprint 2 / 3 Task	Depends On (Sprint 1)
`Game` service / `GameSessionManager`	`GAME-ABSTRACT` (Game interface, GameEngine interface), `PC-EVENTS` (EventBus, GameStateUpdate)
`TicTacToeGame` / `ConnectFourGame`	`GAME-ABSTRACT` (Game, RuleValidator, PlayerAction interfaces)
`clientGame` implementations	`GAME-ABSTRACT` (clientGame interface), game-specific RuleValidator
`Leaderboard` (full)	`GAME-MODEL` (GeneralStats, Rank), `PC-DB` (Database stub), `PC-EVENTS` (GameStateUpdate)
`StatsManager`	`GAME-MODEL` (GeneralStats, Rank), `PC-DB` (Database stub)
All UI screens	`PC-CLIENT` (ClientCore API), `PC-EVENTS` (GameStateUpdate), `GAME-MODEL` (Player)
`EventBus` full wiring	`PC-SESSION` (GameSessionManager), `PC-SERVER` (ServerCore)

Product Backlog

Each ID corresponds to one feature branch (`feat/<ID>`). Branch scope is intentionally sized for a single focused merge request reviewed by the TA before merging into `develop` .

Priority: `High` — required for a working demo. `Medium` — improves completeness. `Low` — deferred if time is short.

ID	Branch	Feature Scope	Priority	Sub-Team
PC-SETUP	`feat/PC-SETUP`	Project package structure (`ui` , `platform` , `game` , `model`)	High	Platform Core
PC-NET	`feat/PC-NET`	`NETWORK` stub — TCP socket `send` / `receive` boundary	High	Platform Core
PC-CLIENT	`feat/PC-CLIENT`	`ClientCore` + `ClientReceiver` (client-side request handling, server-push listener, `clientGame` hook)	High	Platform Core
PC-SERVER	`feat/PC-SERVER`	`ServerCore` — main server router, `routeContent` to all services	High	Platform Core
PC-AUTH	`feat/PC-AUTH`	`SessionManager` + `CredentialsDatabase` stub (login, register, token lifecycle)	High	Platform Core
PC-DB	`feat/PC-DB`	`Database` stub (`storeData` , `retrieveData` — scoreboard, game	High	Platform Core

ID	Branch	Feature Scope	Priority	Sub-Team
		history)		
PC-EVENTS	`feat/PC-EVENTS`	`EventBus` + `GameStateUpdate` event class + `GameStatus` enum	High	Platform Core
PC-LOBBY	`feat/PC-LOBBY`	`GameLobby` service (game info display, chat entry, matchmaking entry point)	High	Platform Core
PC-MATCH	`feat/PC-MATCH`	`matchmaking` service (queue, match-found routing, new `Game` instance creation)	High	Platform Core
PC-CHAT	`feat/PC-CHAT`	`Chat` service (broadcast to user list, `sendMessage`, `receiveContent`)	High	Platform Core
PC-SESSION	`feat/PC-SESSION`	`Game` service + `GameEngine` interface + `GameSessionManager` (full move pipeline, game-over handling)	High	Platform Core
PC-LEADERBOARD	`feat/PC-LEADERBOARD`	`Leaderboard` service (`logGameResult`, `updatePlayer`, `getTopPlayersByRank`, MMR)	Medium	Platform Core
GAME-ABSTRACT	`feat/GAME-ABSTRACT`	`Game`, `RuleValidator`, `PlayerAction`, `clientGame` interfaces/abstractions	High	Rules & Validation
GAME-MODEL	`feat/GAME-MODEL`	`Player`, `GeneralStats`, `PlayerStats`, `StatsManager`, `Rank`, `Settings`	High	Rules & Validation / Platform Core
GAME-TTT	`feat/GAME-TTT`	Tic-Tac-Toe — `TicTacToeGame`, `RuleValidator`, `PlayerAction`, `GameState`, `clientGame` impl, win detection	High	Rules & Validation
GAME-C4	`feat/GAME-C4`	Connect Four — `ConnectFourGame`, `RuleValidator`, `PlayerAction`, `GameState`, `clientGame` impl, win detection	High	Rules & Validation
UI-AUTH	`feat/UI-AUTH`	`WelcomePage`, `AuthenticationScreen`, `LoginPage`, `State` enum	High	Client/UI
UI-HOME	`feat/UI-HOME`	`HomePage`, `MainGameDashboard`, leaderboard view, full navigation wiring	High	Client/UI

ID	Branch	Feature Scope	Priority	Sub-Team
UI-GAME	`feat/UI-GAME`	`PlayGame`, `GameGUI`, `ChatboxGUI`, result screen, turn alert display	High	Client/UI
UI-PROFILE	`feat/UI-PROFILE`	`ProfileDashboard`, `Settings` UI, player statistics view	Medium	Client/UI
QA-SETUP	`feat/QA-SETUP`	Repository, README, branching conventions, test infrastructure, architecture doc maintenance	High	QT&I
QA-TESTS	`feat/QA-TESTS`	JUnit unit tests + integration tests (all flows)	High	QT&I
QA-RELEASE	`feat/QA-RELEASE`	Sprint merges into `develop` / `main`, smoke tests, demo script, final demo run	High	QT&I

Plan version: aligned with architecture document v1.1.0 (updated Platform Core design). Adjust task assignments to individual team members at the start of each sprint.